

# A certified outer bound for convex NLP scenario subproblems

mpi-sppy

companion to `doc/designs/ipopt_outer_bound_design.md`

## Abstract

The usual device for getting outer bounds in mpi-sppy is to read the dual bound off the subproblem solver, which fails for Ipopt, which reports no dual bound. This note derives the bound that the `ipopt_outer_bound` spoke computes instead. The construction is Lagrangian weak duality combined with a tangent-plane underestimator minimised in closed form over the variable box. *None of the mathematics is new*: Theorem 2 is the Frank–Wolfe duality gap on a box, and Section 8 places each result against the literature. The derivation is written out because the hypotheses are what matter in practice, and one of them is easy to get backwards. Its defining property is that it requires *no* assumption about the accuracy of the solve: a truncated or sloppy solve yields a loose bound rather than an invalid one, and an ill-conditioned one does the same, since the construction evaluates rather than solves and so admits no amplification of error by a condition number. Convexity, by contrast, is genuinely load-bearing, and Section 5 isolates the one place where it is easy to get backwards.

## 1 Setting

We assume minimization. Fix a scenario  $s$  with probability  $p_s$ ,  $\sum_s p_s = 1$ . Let  $v \in \mathbb{R}^n$  collect *all* variables of the scenario subproblem — the nonanticipative variables  $x(v)$  together with the recourse variables — and let

$$B = \prod_{i=1}^n [l_i, u_i]$$

be the box of variable bounds. With hub weights  $W_s$ , the subproblem an outer bound spoke solves is

$$L_s(W_s) = \min_v \{ f_s(v) + W_s^\top x(v) : g_s(v) \leq 0, h_s(v) = 0, v \in B \}, \quad (1)$$

with  $f_s : \mathbb{R}^n \rightarrow \mathbb{R}$ ,  $g_s : \mathbb{R}^n \rightarrow \mathbb{R}^m$  and  $h_s : \mathbb{R}^n \rightarrow \mathbb{R}^k$ . This is exactly the problem the ordinary Lagrangian spoke builds, with the proximal term switched off.

**Why the returned objective value is not a bound here.** For a minimisation, the value the solver reports is the objective evaluated at the point it stopped at. Any such value is  $\geq$  the minimum, so it is an *inner* bound: the wrong direction. Nor does Ipopt’s convergence tolerance close the gap, since it is a relative KKT-residual tolerance rather than a bound on the objective error, and the two error sources point opposite ways — optimality error stops slightly above the optimum, while constraint violation can place the iterate slightly below it.

## 2 Weak duality

**Lemma 1** (Weak duality). *Let  $\lambda \in \mathbb{R}_+^m$  and  $\mu \in \mathbb{R}^k$  be arbitrary. Define*

$$\varphi_s(v) = f_s(v) + W_s^\top x(v) + \lambda^\top g_s(v) + \mu^\top h_s(v), \quad (2)$$

$$q_s(\lambda, \mu) = \inf_{v \in B} \varphi_s(v). \quad (3)$$

*Then  $q_s(\lambda, \mu) \leq L_s(W_s)$ .*

*Proof.* Let  $v$  be any point feasible for (1). Then  $v \in B$ , so  $q_s(\lambda, \mu) \leq \varphi_s(v)$ . Moreover  $g_s(v) \leq 0$  and  $\lambda \geq 0$  give  $\lambda^\top g_s(v) \leq 0$ , and  $h_s(v) = 0$  gives  $\mu^\top h_s(v) = 0$ ; hence  $\varphi_s(v) \leq f_s(v) + W_s^\top x(v)$ . Chaining the two inequalities,  $q_s(\lambda, \mu) \leq f_s(v) + W_s^\top x(v)$  for every feasible  $v$ . Taking the infimum over feasible  $v$  gives the claim.  $\square$

Note what Lemma 1 does *not* require:  $\lambda$  and  $\mu$  need not be optimal, or even good. Any  $\lambda \geq 0$  and any  $\mu$  will do. This is the property that makes the whole approach robust, and it is why a mistaken sign convention or a stale multiplier can only cost tightness (Remark 4).

**The trap.** Lemma 1 is not yet usable, because  $q_s$  is defined by an *infimum*. Handing  $\varphi_s$  to a nonlinear solver returns a *point*, and the value there is  $\geq$  the infimum — an upper bound on  $q_s$ , which is the wrong direction again. A second NLP solve therefore does not by itself certify anything.

## 3 The box underestimator

**Theorem 2** (Certified bound). *Suppose  $\varphi_s$  is convex on an open set containing  $B$  and differentiable at  $\hat{v}$ . Define*

$$\hat{q}_s(\hat{v}) = \varphi_s(\hat{v}) + \sum_{i=1}^n \inf_{t \in [l_i, u_i]} \partial_i \varphi_s(\hat{v}) (t - \hat{v}_i). \quad (4)$$

*Then  $\hat{q}_s(\hat{v}) \leq q_s(\lambda, \mu) \leq L_s(W_s)$ , and each term of the sum is available in closed form:*

$$\inf_{t \in [l_i, u_i]} \partial_i \varphi_s(\hat{v}) (t - \hat{v}_i) = \begin{cases} \partial_i \varphi_s(\hat{v}) (l_i - \hat{v}_i), & \partial_i \varphi_s(\hat{v}) > 0, \\ \partial_i \varphi_s(\hat{v}) (u_i - \hat{v}_i), & \partial_i \varphi_s(\hat{v}) < 0, \\ 0, & \partial_i \varphi_s(\hat{v}) = 0. \end{cases} \quad (5)$$

*The right-hand side is read in the extended reals: a term is  $-\infty$  exactly when  $l_i = -\infty$  with  $\partial_i \varphi_s(\hat{v}) > 0$ , or  $u_i = +\infty$  with  $\partial_i \varphi_s(\hat{v}) < 0$ . The bound is then vacuous rather than invalid; Section 7 says what the implementation does with it.*

*Proof.* Convexity and differentiability at  $\hat{v}$  give the gradient inequality

$$\varphi_s(v) \geq \varphi_s(\hat{v}) + \nabla \varphi_s(\hat{v})^\top (v - \hat{v}) \quad \text{for all } v.$$

Minimising both sides over  $v \in B$  preserves the inequality, and the right-hand side is separable across coordinates because  $B$  is a box, which yields (4). Each one-dimensional problem minimises a linear function over an interval. When the endpoint the sign of  $\partial_i \varphi_s(\hat{v})$  selects is finite the infimum is attained there; when it is infinite the infimum is  $-\infty$ . Either way (5) holds, read in the extended reals, and the final inequality is Lemma 1.  $\square$

**Corollary 3.**  $\hat{q}_s(\hat{v}) \leq L_s(W_s)$  for any  $\hat{v}$  at which  $\varphi_s$  is differentiable, any  $\lambda \geq 0$  and any  $\mu$ . In particular  $\hat{v}$  need not be optimal, and need not even be feasible for (1).

Corollary 3 is the whole point. The certificate consumes the iterate the solver happened to stop at and the multipliers it happened to report, and returns a valid bound regardless. Computationally it costs one gradient evaluation and a loop over the variables — no second solve. It is worth reading the corollary as covering *ill-conditioned* solves and not merely truncated ones; Remark 9 says why the distinction does not arise here.

*Remark 4 (Robustness).* Because Corollary 3 holds for arbitrary  $\lambda \geq 0$  and  $\mu$ , an error in the sign convention used to recover the multipliers, a stale multiplier, or a multiplier clipped to zero can only make  $\hat{q}_s$  smaller. Such an error makes the bound loose, never invalid. Convexity enjoys no such protection; see Section 5.

## 4 Exactness at a KKT point

The correction term in (4) is not merely small in practice: it vanishes identically at an exact KKT point, so the certificate degrades gracefully and costs nothing when the solve is good.

**Proposition 5** (The correction vanishes). *Let  $\hat{v}$  satisfy the KKT conditions of (1) with multipliers  $\lambda \geq 0$  for  $g_s \leq 0$ ,  $\mu$  for  $h_s = 0$ , and  $z_L, z_U \geq 0$  for the bounds  $l - v \leq 0$  and  $v - u \leq 0$ . Then every term of the sum in (4) is zero, and if in addition  $\varphi_s$  is convex then  $\hat{q}_s(\hat{v}) = L_s(W_s)$ .*

*Proof.* Stationarity of the full Lagrangian, which is  $\varphi_s$  augmented with the bound terms, reads

$$\nabla \varphi_s(\hat{v}) = z_L - z_U. \quad (6)$$

Fix a coordinate  $i$  and consider the three possible positions of  $\hat{v}_i$ .

If  $l_i < \hat{v}_i < u_i$ , complementarity gives  $z_{L,i} = z_{U,i} = 0$ , so  $\partial_i \varphi_s(\hat{v}) = 0$  by (6) and the term is 0 by (5).

If  $\hat{v}_i = l_i$ , complementarity gives  $z_{U,i} = 0$ , so  $\partial_i \varphi_s(\hat{v}) = z_{L,i} \geq 0$ . When the derivative is strictly positive (5) selects  $t = l_i = \hat{v}_i$ , and when it is zero the term is zero outright; either way the term is 0.

If  $\hat{v}_i = u_i$ , symmetrically  $z_{L,i} = 0$  and  $\partial_i \varphi_s(\hat{v}) = -z_{U,i} \leq 0$ , so (5) selects  $t = u_i = \hat{v}_i$  and the term is 0.

Hence  $\hat{q}_s(\hat{v}) = \varphi_s(\hat{v})$ . Complementarity  $\lambda^\top g_s(\hat{v}) = 0$  and feasibility  $h_s(\hat{v}) = 0$  reduce (2) to  $\varphi_s(\hat{v}) = f_s(\hat{v}) + W_s^\top x(\hat{v})$ . Under convexity the KKT conditions are sufficient for global optimality of (1), so that value is  $L_s(W_s)$ .  $\square$

Proposition 5 has a practical corollary worth stating plainly: the correction term *measures its own inexactness*. It is zero when the solve is exact and grows as the solve degrades, so the reported bound loosens smoothly rather than becoming wrong.

## 5 Canonical form, and where convexity is easy to get backwards

Theorem 2 requires  $\varphi_s$  convex, and by (2) that requires each component of  $g_s$  to be convex and each component of  $h_s$  to be affine, since  $\lambda \geq 0$  but  $\mu$  is free in sign.

The subtlety is that  $g_s$  is the constraint in *canonical* form  $g_s(v) \leq 0$ , and putting a  $\geq$  row into canonical form negates its body. Writing  $b(v)$  for the constraint body as the user stated it:

| as written                    | canonical form   | requirement on $b$ |
|-------------------------------|------------------|--------------------|
| $b(v) \leq \beta$             | $g = b - \beta$  | $b$ convex         |
| $b(v) \geq \alpha$            | $g = \alpha - b$ | $b$ <b>concave</b> |
| $\alpha \leq b(v) \leq \beta$ | both rows        | $b$ affine         |
| $b(v) = \beta$                | $h = b - \beta$  | $b$ affine         |

So  $x^2 \leq 4$  is admissible and  $x^2 \geq 1$  is not, though both are written with a convex body — and indeed the feasible set of the second is not convex. This is the one place in the construction where an entirely ordinary-looking model silently leaves the hypotheses, and the consequence is not a loose bound but an invalid one. A worked instance: for

$$\min x \quad \text{s.t.} \quad x^2 \geq 1, \quad x \in [0, 1.5],$$

whose optimum is 1, taking  $\hat{v} = 0.5$  and  $\lambda = 1$  in (4) yields  $\hat{q} = 1.25 > 1$ .

Of the four rows above, *two* are decidable — the two that demand an affine body, since affineness is a question about polynomial degree — and the implementation enforces those two. The other two rows are the one-sided ones, and each asks a question that is not decidable here: whether the body of a  $\leq$  row is convex, and whether the body of a  $\geq$  row is concave. Both remain user assertions, and the second is the trap above — a  $\geq$  row with a convex body passes every mechanical check and breaks the theorem.

## 6 Aggregating across scenarios

The per-scenario bounds are combined by the usual probability-weighted sum, and the condition under which that sum is itself a bound deserves care.

**Proposition 6** (Aggregation). *Let  $\text{OPT}$  be the optimal value of the original stochastic program. If the weights satisfy  $\sum_s p_s W_s = 0$ , then  $\sum_s p_s L_s(W_s) \leq \text{OPT}$ .*

*Proof.* Let  $x^*$  together with the scenario recourse decisions be optimal for the original problem, so that  $\text{OPT} = \sum_s p_s f_s(v_s^*)$  where  $v_s^*$  is the optimal decision in scenario  $s$  and  $x(v_s^*) = x^*$  for every  $s$  by nonanticipativity. Each  $v_s^*$  is feasible for subproblem (1), so  $L_s(W_s) \leq f_s(v_s^*) + W_s^\top x^*$ . Multiplying by  $p_s$  and summing,

$$\sum_s p_s L_s(W_s) \leq \sum_s p_s f_s(v_s^*) + \left( \sum_s p_s W_s \right)^\top x^* = \text{OPT} + 0. \quad \square$$

*Remark 7* (Weights must share a generation). The hypothesis  $\sum_s p_s W_s = 0$  is a *joint* condition on the whole weight vector, and progressive hedging maintains it at each iteration. It is not preserved by mixing: if  $W'_s$  takes scenario  $s$ 's weights from one iteration and scenario  $t$ 's from another, then in general  $\sum_s p_s W'_s \neq 0$ , the proof above leaves the residual term  $(\sum_s p_s W'_s)^\top x^*$  in place, and the resulting number is not a bound at all. Nothing downstream can detect this, which is why a scenario whose certificate is unavailable must make the whole expectation unavailable rather than contribute a stale value.

## 7 Practical consequences

**Tightness is governed by the box.** By (5) the shortfall of  $\hat{q}_s$  below  $\varphi_s(\hat{v})$  is

$$\sum_{i=1}^n |\partial_i \varphi_s(\hat{v})| \cdot (\text{distance from } \hat{v}_i \text{ to the far end of } [l_i, u_i]),$$

so the bound is only as tight as the variable bounds are. Gradient components at a converged solve are small but generically nonzero, so a box of width 10 costs little while a box of width  $10^{10}$  can cost a substantial fraction of the objective. Tightening bounds before certifying — for instance by feasibility-based bounds tightening, which shrinks  $B$  without removing feasible points and therefore only raises  $\hat{q}_s$  — is the most effective way to improve the bound.

*Remark 8* (Admissible tightenings of the box). The box in (5) need not be the one the model was written with, and the implementation shrinks it by two different mechanisms. The condition under which this is legitimate is not that a smaller box removes points — that argument shows the bound gets tighter, which is the direction that could break it — but the following. Let  $x^*$  be an optimal solution of the full problem. If  $x^* \in B_s$  for every  $s$ , then for each  $s$

$$\hat{q}_s \leq \inf_{v \in B_s} \varphi_s(v) \leq \varphi_s(x^*) \leq f_s(x^*) + W_s^\top x^*,$$

the last step because  $\lambda \geq 0$ ,  $g_s(x^*) \leq 0$  and  $h_s(x^*) = 0$ , and Proposition 6 goes through unchanged. The requirement is therefore that *every  $B_s$  contain one common optimal solution of the full problem*; preserving a different optimiser in each scenario does not suffice, since the weighted sum telescopes only at a single  $x^*$ .

Two sufficient conditions are used. Feasibility-based bounds tightening satisfies the requirement in the strong form: it removes no point satisfying the constraints of subproblem  $s$ , so  $B_s$  retains every feasible point of (1) and in particular  $x^*$ , with no appeal to optimality. Optimality-based tightening — reduced-cost fixing, which does discard feasible points — satisfies only the weak form, and relies on its own contract that some optimal solution survives, together with the fact that the same bounds are applied to every scenario, so that the surviving solution is common to all of them. A tightening that guarantees neither is not admissible, and no check in the implementation can detect that.

**Unbounded variables.** If  $l_i = -\infty$  and  $\partial_i \varphi_s(\hat{v}) > 0$ , or  $u_i = +\infty$  and  $\partial_i \varphi_s(\hat{v}) < 0$ , the corresponding term in (5) is  $-\infty$  and no finite bound is available. Whether this occurs depends on the model and on how converged the solve is: at an exact KKT point the offending component is zero by Proposition 5, but away from one it generally is not. The implementation reports “no bound” in that case rather than  $-\infty$ , so that Remark 7 is respected.

**Inexact solves.** Corollary 3 already covers these, but it is worth spelling out what “inexact” can mean. Problem (1) is convex by hypothesis, so it has no non-global local minima. A solver returning a sub-optimal answer can therefore only mean that it stopped short of converging, leaving an inexact  $\hat{v}$  and inexact multipliers. Both are admissible inputs to Theorem 2: the conclusion  $\hat{q}_s \leq L_s(W_s)$  is unchanged, and the entire effect appears in the correction term of (4), which grows as  $\hat{v}$  moves away from a KKT point. By Proposition 5 that term vanishes exactly at a KKT point, so the correction is precisely the price of inexactness, and it measures itself.

*Remark 9 (Conditioning).* The distinct worry is that (1) is badly conditioned, so that the solve is not merely truncated but numerically poor. This affects the tightness of  $\hat{q}_s$  and not its validity, for a structural reason: *the certificate is an evaluation, not a solve*. A condition number quantifies how much a linear solve amplifies error —  $\kappa(H)$  bounds the relative error in  $d$  obtained from  $Hd = -g$  — and no step of (5) inverts anything. The computation evaluates  $\varphi_s$  at  $\hat{v}$ , evaluates  $\nabla\varphi_s(\hat{v})$ , compares each component with zero, selects an endpoint of the corresponding interval, multiplies and adds. Each is a forward evaluation, and  $\kappa$  has no route in. Correspondingly, the inequality the construction rests on,

$$\varphi_s(v) \geq \varphi_s(\hat{v}) + \nabla\varphi_s(\hat{v})^\top(v - \hat{v}) \quad \text{for all } v,$$

is a pointwise consequence of convexity. It holds exactly, at every  $\hat{v}$ , with no error constant and no dependence on the conditioning of anything. Conditioning governs how loose the plane is away from  $\hat{v}$ ; it cannot change which side of  $\varphi_s$  the plane lies on. What it does change is the size of  $\nabla\varphi_s(\hat{v})$  and the quality of the multipliers, and both push  $\hat{q}_s$  *down*.

**Floating point.** Nothing in Theorem 2 requires a tolerance. The arithmetic, being carried out in finite precision, does introduce error, and the previous paragraph says only that this error is not amplified by  $\kappa$  — not that it is absent.

*Remark 10 (Rounding).* Let  $u$  denote the unit roundoff. Evaluating (4) in floating point incurs

$$|\text{fl}(\hat{q}_s) - \hat{q}_s| \lesssim u \left( |f_s(\hat{v})| + \sum_i \lambda_i |g_i(\hat{v})| + \sum_j |\mu_j| |h_j(\hat{v})| + \sum_i T_i d_i \right),$$

where  $T_i$  bounds the intermediate magnitudes arising in  $\partial_i\varphi_s(\hat{v})$  and  $d_i$  is the distance from  $\hat{v}_i$  to the far end of  $[l_i, u_i]$ . No condition number appears.

Remark 10 is governed by the size of the *summands*, so cancellation matters: multipliers of order  $10^8$  against an objective of order one put the error floor near  $10^{-8}$  even though  $|\hat{q}_s|$  is of order one. The implementation’s cushion, reporting  $\hat{q}_s - \varepsilon(1 + |\hat{q}_s|)$ , is scaled to the size of the *result* instead, and the two agree only on a well-scaled model. The cushion is therefore a numerical safeguard and not part of the argument, and it is exposed as a user-settable quantity rather than fixed.

Note also that the final term of Remark 10 is the one that can raise  $\hat{q}_s$ . The correction in (5) equals  $-\sum_i |\partial_i\varphi_s(\hat{v})| d_i$ , so a gradient component computed slightly small in magnitude makes it slightly less negative. Remark 4 gives no protection here: it is a statement about the multipliers, and a perturbation of  $\nabla\varphi_s$  is not one-directional in the way a perturbation of  $\lambda$  is.

**Non-finite values.** A solve that diverges can leave NaN in  $\hat{v}$  or in the reported multipliers, and an infinite multiplier yields NaN through  $\infty \cdot 0$  or an infinite  $\varphi_s(\hat{v})$ . These propagate silently. The implementation therefore rejects any non-finite  $\hat{q}_s$  and reports “no bound”, the same disposition it takes for an unbounded direction, so that Remark 7 is again respected. The case that makes this necessary rather than cosmetic is  $+\infty$ : a consumer keeping the best outer bound seen would discard NaN by accident, since NaN fails every comparison, but would accept  $+\infty$  as an improvement.

## 8 Relation to known results

Nothing in Sections 2–7 is new. This section attributes each result.

**Lemma 1** is textbook Lagrangian weak duality, stated for an arbitrary multiplier pair rather than an optimal one.

**Theorem 2** is the *Frank–Wolfe duality gap*, sometimes called the Wolfe gap or the linearisation gap. For a convex  $f$  on a compact convex  $D$ , evaluating the linear minimisation oracle at  $x$  gives

$$f(x) + \min_{y \in D} \nabla f(x)^\top (y - x) \leq \min_D f,$$

which is (4) with  $D = B$ . The bound is used in the Frank–Wolfe algorithm [1], where the same oracle evaluation that produces it also supplies the search direction; reading it instead as a *certificate* computable at any iterate is the framing in [2]. The only specialisation here is that a box makes the oracle separable and closed-form, so the certificate costs one gradient evaluation and no optimisation at all.

**Proposition 6** is the standard argument by which progressive hedging weights yield a lower bound on the optimal value, as in [3].

**The remarks of Section 7** are elementary rather than novel. Remark 10 is the textbook forward-error bound for a floating-point sum, specialised to the summands of (4), and Remark 8 is two applications of Lemma 1. They are stated because the hypotheses they turn on are easy to lose sight of in implementation, not because they are results.

**The combination** of progressive hedging with Frank–Wolfe machinery to obtain Lagrangian dual bounds is itself established [4], and mpi-sppy already ships an FWPH cylinder on that basis.

What is specific here is therefore an assembly rather than a result: the multipliers are the ones Ipopt already returns, the linearisation point is the iterate it already stopped at, and the box is the model’s own variable bounds — so a valid outer bound is obtained from the solve the spoke was going to perform anyway, with no second optimisation and no assumption that the first one converged. The wider question of extracting valid dual bounds from an inexact oracle is treated at length in the inexact bundle-method literature, which this note does not attempt to survey.

## 9 Correspondence with the implementation

| object here                 | in the code                                        |
|-----------------------------|----------------------------------------------------|
| $\varphi_s$ in (2)          | <code>_lagrangian_expression</code>                |
| $\hat{q}_s$ in (4)          | <code>certified_lower_bound</code>                 |
| decidable hypotheses        | <code>check_model_is_certifiable</code>            |
| box tightening, strong form | <code>unbounded_variables (fbbt)</code>            |
| Remark 10 cushion           | <code>eps_rel</code>                               |
| non-finite rejection        | <code>math.isfinite</code> test on $\hat{q}_s$     |
| box tightening, weak form   | <code>receive_nonant_bounds</code>                 |
| Proposition 6 sum           | <code>SPOpt.Ebound</code>                          |
| Remark 7 safeguard          | a <code>None</code> bound, propagated collectively |

Everything above the rule is in `utils/dual_certificate.py`, whose `eps_rel` argument is exposed on the command line as `--ipopt-outer-bound-cushion`. Below it, the weak-form tightening is in `cylinders/spcommunicator.py` and the sum is in `spopt.py`, with the `None` propagated across both. All paths are relative to `mpisppy/`, and the spoke driving the whole is `cylinders/ipopt_outer_bound.py`.

## A Why the reduced costs are not a shortcut

Ipopt reports bound multipliers, and a natural proposal is to read the distance below the returned objective value off them directly, saving the gradient evaluation that Theorem 2 calls for. This appendix records why the implementation does not do that. The short version is that the reduced costs are not an alternative route to the bound — they are the same quantity the certificate already uses — and that substituting the reported values for the gradient forfeits validity in a way that is systematic rather than occasional.

### A.1 The reduced cost is the gradient

Equation (6) in the proof of Proposition 5 is exactly the identification: at a KKT point,  $\nabla\varphi_s(\hat{v}) = z_L - z_U$ , so the reduced cost of variable  $i$  is  $\partial_i\varphi_s(\hat{v})$ , and the distance below  $\varphi_s(\hat{v})$  that must be conceded is the correction term of (4). Specialising  $\varphi_s$  to a linear function turns (4) into the textbook reduced-cost dual bound of linear programming,

$$c^\top \hat{x} + \sum_j \min\{d_j(l_j - \hat{x}_j), d_j(u_j - \hat{x}_j)\},$$

with  $d$  the vector of reduced costs; (4) is that bound’s nonlinear generalisation. So the proposal is not a cheaper substitute for Theorem 2. It is Theorem 2, written in multiplier variables instead of gradient variables. In particular there is no second solve to be avoided: Corollary 3 already costs one gradient evaluation and a loop over the variables.

What a reduced-cost implementation would genuinely save is that gradient evaluation, replaced by a read of the solver’s reported bound multipliers. That is a real cost — it is an automatic-differentiation sweep over  $\varphi_s$  — but it is the cost of a gradient, not of a solve.

### A.2 Why the substitution is not valid

Equation (6) is a property of an exact KKT point, which is a hypothesis of Proposition 5 and not a fact about an arbitrary iterate. At a truncated solve the two sides differ by the dual-infeasibility residual

$$r = \nabla\varphi_s(\hat{v}) - (z_L - z_U) \neq 0. \quad (7)$$

Theorem 2 rests on the gradient inequality, which holds for a true (sub)gradient of a convex function and for nothing else. Replacing  $\nabla\varphi_s(\hat{v})$  by  $z_L - z_U$  therefore removes the hypothesis the proof uses, and Remark 4 offers no cover: that remark is a statement about  $\lambda$  and  $\mu$ , whereas this is a perturbation of the gradient, which Remark 10 already identifies as the one perturbation that can raise  $\hat{q}_s$ .



### A.3 The failure is systematic, not occasional

The substitution does not merely risk an invalid bound now and then. It drives the correction term to zero by construction, at every iterate, however unconverged.

**Proposition 11** (The substituted correction collapses). *Let  $\hat{v}$  satisfy the perturbed complementarity conditions of the barrier subproblem with parameter  $\tau > 0$ , that is  $z_{L,i}(\hat{v}_i - l_i) = \tau$  for every finite  $l_i$  and  $z_{U,i}(u_i - \hat{v}_i) = \tau$  for every finite  $u_i$ , with  $z_{L,i} = 0$  when  $l_i = -\infty$  and  $z_{U,i} = 0$  when  $u_i = +\infty$ . Substitute  $d_i = z_{L,i} - z_{U,i}$  for  $\partial_i \varphi_s(\hat{v})$  in (5). Then every term of the resulting sum lies in  $(-\tau, 0]$ , so the substituted correction lies in  $(-n\tau, 0]$  and*

$$\hat{q}_s^{\text{rc}}(\hat{v}) \geq \varphi_s(\hat{v}) - n\tau.$$

*Proof.* Fix  $i$  and write  $\sigma = \hat{v}_i - l_i > 0$  and  $\theta = u_i - \hat{v}_i > 0$  in the two-sided case. Then  $d_i = \tau/\sigma - \tau/\theta$ , so  $d_i > 0$  exactly when  $\sigma < \theta$ . In that case (5) selects  $t = l_i$  and the term is

$$d_i(l_i - \hat{v}_i) = -\sigma \left( \frac{\tau}{\sigma} - \frac{\tau}{\theta} \right) = -\tau \left( 1 - \frac{\sigma}{\theta} \right) \in (-\tau, 0),$$

using  $0 < \sigma < \theta$ . The case  $d_i < 0$  is symmetric with  $\sigma$  and  $\theta$  exchanged, and  $d_i = 0$  gives the term 0. If  $u_i = +\infty$  then  $z_{U,i} = 0$  and  $d_i = \tau/\sigma > 0$ , so (5) selects  $t = l_i$  and the term is exactly  $-\tau$ ; the case  $l_i = -\infty$  is symmetric. Summing over  $i$  gives the claim.  $\square$

Two consequences make the proposal worse than merely unsound.

First, the quantity returned is governed by the solver’s barrier parameter and not by the inexactness of the solve. As  $\tau \rightarrow 0$  it converges to  $\varphi_s(\hat{v})$ , which under complementarity and feasibility is the returned objective value — the inner bound ruled out under *Why the returned objective value is not a bound here* in Section 1. A scheme that reports it is not a certificate; it is the number the certificate was built to replace, shaved by  $n\tau$ . What is lost is precisely the property recorded under *Inexact solves* in Section 7: that the correction term measures its own inexactness and grows as  $\hat{v}$  moves away from a KKT point.

Second, the substitution silently defeats the unbounded-variable guard. When  $u_i = +\infty$  the reported  $z_{U,i}$  is zero, so  $d_i \geq 0$  and (5) never selects the infinite endpoint. The  $-\infty$  that the honest gradient produces for an unconverged component with a missing bound — the case discussed under *Unbounded variables* in Section 7, which the implementation reports as “no bound” — can never arise, so a finite value would be returned in exactly the situation where no finite bound is available.

### A.4 A valid variant, and why it is dominated

Validity can be restored by paying for the residual (7) explicitly. For any  $t \in B$ ,  $|r^\top(t - \hat{v})| \leq \|r\|_\infty \|t - \hat{v}\|_1 \leq \|r\|_\infty \|\text{diam}(B)\|_1$ , so subtracting  $\varepsilon_d \|\text{diam}(B)\|_1$  from  $\hat{q}_s^{\text{rc}}$  restores a valid bound whenever  $\|r\|_\infty \leq \varepsilon_d$ . Two cautions attach: the box must have finite diameter, which Section 7 shows is not guaranteed, and  $\varepsilon_d$  must be the *unscaled* dual infeasibility, since the reported convergence measure is scaled by default.

Even so the variant is dominated in both regimes. Near a KKT point the honest correction is already zero by Proposition 5, so there is no tightness to be gained and the safeguard can only subtract. Away from one, the safeguard is loose exactly where the gradient form is loose but correct. The gradient form is at least as tight everywhere, and it needs no estimate of  $\|r\|_\infty$ .

## A.5 Where a saving would actually come from

The cost the proposal targets is real, and it is not negligible: rebuilding  $\varphi_s$  and walking it takes about 3 seconds at  $10^5$  constraint rows, growing linearly at roughly  $30 \mu\text{s}$  per row, so a million-row scenario would spend on the order of 30 seconds per iteration inside the certificate. (A figure from one machine, quoted for its order of magnitude.) But it is the cost of differentiating  $\varphi_s$ , and it is addressable without touching the mathematics. The remedy is a faster *exact* gradient — forming  $\nabla f_s + W_s + \nabla g_s^\top \lambda + \nabla h_s^\top \mu$  through compiled automatic differentiation rather than by walking a Python expression tree, or caching the expression skeleton across iterations with the multipliers held in mutable parameters — either of which leaves every hypothesis of Theorem 2 intact. Cheapening the gradient is safe; approximating it is not.

## References

- [1] Frank, M. and Wolfe, P.: An algorithm for quadratic programming. *Naval Research Logistics Quarterly* **3**(1–2), 95–110 (1956).
- [2] Jaggi, M.: Revisiting Frank–Wolfe: projection-free sparse convex optimization. *Proceedings of the 30th International Conference on Machine Learning (ICML)*, 427–435 (2013).
- [3] Gade, D., Hackebeil, G., Ryan, S.M., Watson, J.-P., Wets, R.J.-B. and Woodruff, D.L.: Obtaining lower bounds from the progressive hedging algorithm for stochastic mixed-integer programs. *Mathematical Programming* **157**(1), 47–67 (2016).
- [4] Boland, N., Christiansen, J., Dandurand, B., Eberhard, A., Linderoth, J., Luedtke, J. and Oliveira, F.: Combining progressive hedging with a Frank–Wolfe method to compute Lagrangian dual bounds in stochastic mixed-integer programming. *SIAM Journal on Optimization* **28**(2), 1312–1336 (2018).